



AMRITA
VISHWA VIDYAPEETHAM

School of Engineering
Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

Agneay B Nair
CH.SC.U4CSE24102

WORKSHEET



WORKSHEET-1

EX.NO : 1

DATE : 18/06/2026

TITLE: INTRODUCTION TO BITS, GATES AND QUANTUM CIRCUITS

AIM:

To study the fundamental concepts of quantum computing, including classical bits, quantum bits (qubits), quantum states, superposition, quantum gates, and quantum circuit representation. The objective is to understand how quantum information is represented and manipulated using various quantum gates and circuit diagrams, which form the foundation of quantum computation.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit Quantum Computing Framework
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

INSTALLATION:

- Step 1: Install Python
- Download and install Python 3.10 or later from the official Python website.
- During installation, select the option "Add Python to PATH".
- Step 2: Verify Python Installation
- Open Command Prompt and execute:
- `python --version`
- The installed Python version should be displayed.



- Step 3: Create a Virtual Environment
- Create a new virtual environment for Qiskit:
- `python -m venv qiskit-env`
- Activate the environment:
- `qiskit-env\Scripts\activate`
- Step 4: Upgrade pip
- Upgrade the package installer using:
- `python -m pip install --upgrade pip`
- Step 5: Install Qiskit
- Install the Qiskit framework using:
- `pip install qiskit`
- Step 6: Install Qiskit Aer Simulator
- Install the Aer simulator package:
- `pip install qiskit-aer`
- Step 7: Verify Qiskit Installation
- Execute the following command:
- `python -c "import qiskit; print(qiskit.__version__)"`
- If the Qiskit version is displayed, the installation is successful.
- Step 8: Test Qiskit
- Run the following Python code:
- `from qiskit import QuantumCircuit`
- `qc = QuantumCircuit(2)`
- `qc.h(0)`
- `qc.cx(0,1)`
- `print(qc)`
- The quantum circuit diagram will be displayed, confirming successful installation and



execution of Qiskit.

THEORY:

Introduction to Quantum Computing:

Quantum computing is a revolutionary computing paradigm that leverages the principles of quantum mechanics to process information. Unlike classical computers that operate using bits, quantum computers utilize quantum bits (qubits), enabling enhanced computational capabilities for specific classes of problems.

Classical Bits and Quantum Bits:

A classical bit can exist only in one of two possible states, namely 0 or 1. In contrast, a qubit can exist in the state $|0\rangle$, $|1\rangle$, or a superposition of both states simultaneously, allowing quantum systems to represent and process information more efficiently.

Mathematical Representation of a Qubit:

The state of a qubit is represented as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where α and β are complex probability amplitudes satisfying the normalization condition:

$$|\alpha|^2 + |\beta|^2 = 1$$

Quantum Circuit Model:

Quantum computations are represented using quantum circuits. A quantum circuit consists of qubits, quantum gates, and measurement operations. Information flows from left to right, with quantum gates manipulating the states of qubits throughout the computation process.

Quantum Gates:

Quantum gates are reversible operations that alter the state of qubits. They serve as the fundamental building blocks of quantum algorithms. Commonly used quantum gates include:

- Pauli-X Gate (X)
- Pauli-Y Gate (Y)
- Pauli-Z Gate (Z)
- Hadamard Gate (H)
- Phase Gate (S)



- T Gate
- Controlled-NOT Gate (CNOT)
- Quantum Superposition

Superposition is a fundamental property of quantum systems that allows a qubit to exist in multiple states simultaneously. The Hadamard gate is commonly used to generate superposition states from computational basis states.

Quantum Entanglement:

Entanglement is a unique quantum phenomenon in which two or more qubits become correlated such that the state of one qubit depends on the state of another, regardless of the distance between them. The CNOT gate is frequently used to create entangled states.

Qiskit Framework:

Qiskit is an open-source quantum computing software development framework developed by IBM. It enables users to design, simulate, visualize, and execute quantum circuits on both simulators and real quantum hardware.

Quantum Circuit Simulation:

Qiskit provides simulation tools such as Aer Simulator for testing and validating quantum circuits in a classical computing environment before deploying them on actual quantum processors.

Measurement of Quantum States:

Measurement is the process of converting quantum information into classical information. Upon measurement, a qubit collapses to one of its basis states, and the outcome is recorded as classical data.

Significance of the Experiment

The study of bits, qubits, quantum gates, and quantum circuits using Qiskit provides the fundamental knowledge required for understanding advanced quantum algorithms and quantum information processing systems.

PROCEDURE:

1. Install Python software and configure the Python environment.
2. Install the Qiskit framework and required dependencies.
3. Verify the successful installation of Qiskit.



4. Open Jupyter Notebook or Visual Studio Code.
5. Import the required Qiskit modules.
6. Create a quantum circuit using the QuantumCircuit class.
7. Initialize qubits and classical bits in the circuit.
8. Apply basic quantum gates such as Pauli-X and Hadamard gates.
9. Construct a simple quantum circuit and visualize it.
10. Execute the quantum circuit using the Aer Simulator.
11. Measure the qubits and obtain the output results.
12. Observe and analyze the behavior of bits, qubits, gates, and quantum circuits.

PROGRAM:

1. Create a quantum circuit with a single qubit and display the circuit diagram.
2. Write a Qiskit program to apply the Pauli-X (NOT) gate on a qubit and observe the circuit.
3. Create a quantum circuit using the Hadamard gate and study the concept of superposition.
4. Write a Qiskit program to create a two-qubit circuit and apply a Controlled-NOT (CNOT) gate.
5. Construct a Bell State circuit using Hadamard and CNOT gates.
6. Write a Qiskit program to measure a single qubit and display the output.
7. Create a quantum circuit with two qubits and perform measurements on both qubits.
8. Design and visualize a quantum circuit containing X, H, and CNOT gates.
9. Simulate a quantum circuit using the Aer Simulator and analyze the measurement results.
10. Compare the behavior of classical bits and quantum bits using simple Qiskit circuits.

OUTPUT:

The following ten programs were implemented and executed using Qiskit on the AerSimulator / Statevector backend. For each program the source code, the console output, and the resulting circuit

diagram / measurement screenshot are reproduced below, exactly as obtained from the corresponding Jupyter notebook.

Program 1: Single Qubit Quantum Circuit

Aim:

To create a quantum circuit with a single qubit and display the circuit diagram.

Code:

Import required libraries

```
from qiskit import QuantumCircuit
```

Create a single-qubit quantum circuit

```
# Create a quantum circuit with 1 qubit
qc = QuantumCircuit(1)

# Display the circuit diagram (text form)
print(qc.draw())
```

```
q:
```

Display the circuit diagram graphically

```
qc.draw('mpl')
```

q —

Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed output verifies the expected quantum behaviour described in the aim of this experiment.

Program 2: Pauli-X (NOT) Gate on a Qubit

Aim:

To write a Qiskit program to apply the Pauli-X (NOT) gate on a qubit and observe the circuit.

Code:

Import required libraries

```
from qiskit import QuantumCircuit
```

Create a circuit and apply the Pauli-X gate

```
# Create a quantum circuit with 1 qubit
qc = QuantumCircuit(1)

# Apply the Pauli-X (NOT) gate on qubit 0
qc.x(0)

# Display the circuit diagram
print(qc.draw())
```

q: 

Display the circuit diagram graphically

```
qc.draw('mpl')
```



Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed output verifies the expected quantum behaviour described in the aim of this experiment.

Program 3: Hadamard Gate and Superposition

Aim:

To create a quantum circuit using the Hadamard gate and study the concept of superposition.

Code:

Import required libraries

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
```

Create a circuit and apply the Hadamard gate

```
# Create a quantum circuit with 1 qubit
qc = QuantumCircuit(1)

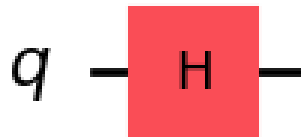
# Apply the Hadamard gate to put the qubit into superposition
qc.h(0)

# Display the circuit diagram
print(qc.draw())
```

q: 

Display the circuit diagram graphically

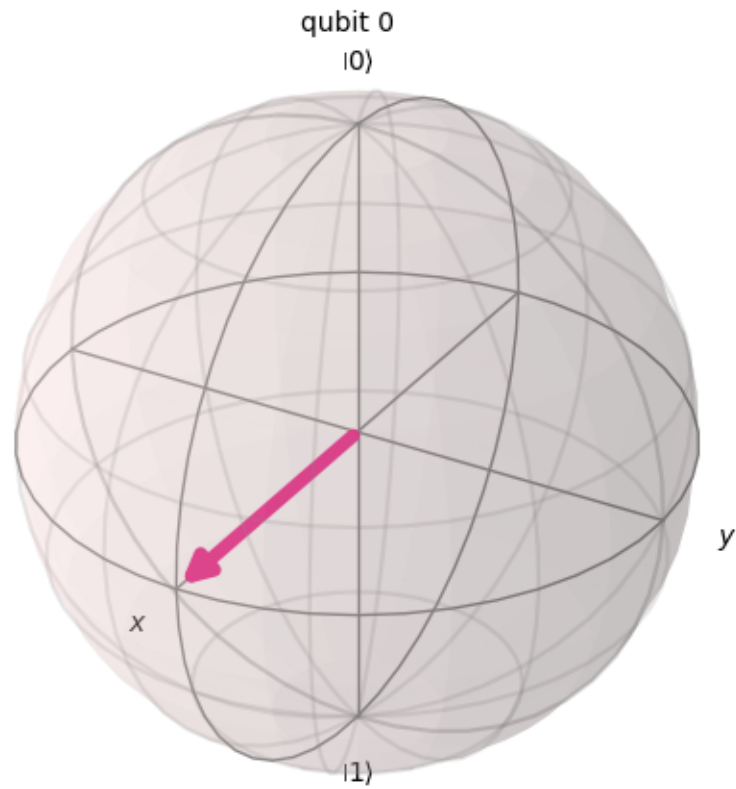
```
qc.draw('mpl')
```



Observe the resulting statevector (superposition)

```
state = Statevector.from_instruction(qc)
print("Statevector:", state)
state.draw('bloch')
```

```
Statevector: Statevector([0.70710678+0.j, 0.70710678+0.j],
                        dims=(2,))
```



Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed output verifies the expected quantum behaviour described in the aim of this experiment.

Program 4: Two-Qubit Circuit with CNOT Gate

Aim:

To write a Qiskit program to create a two-qubit circuit and apply a Controlled-NOT (CNOT) gate.

Code:

Import required libraries

```
from qiskit import QuantumCircuit
```

Create a two-qubit circuit and apply the CNOT gate

```
# Create a quantum circuit with 2 qubits
qc = QuantumCircuit(2)

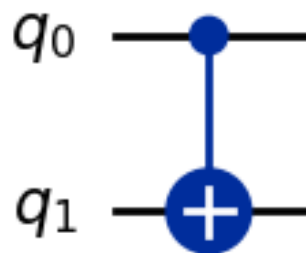
# Apply CNOT gate: qubit 0 = control, qubit 1 = target
qc.cx(0, 1)

# Display the circuit diagram
print(qc.draw())
```

q_0:  —
q_1: 

Display the circuit diagram graphically

```
qc.draw('mpl')
```



Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed output verifies the expected quantum behaviour described in the aim of this experiment.

Program 5: Bell State Circuit (Entanglement)

Aim:

To construct a Bell State circuit using Hadamard and CNOT gates and study quantum entanglement.

Code:

Import required libraries

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
```

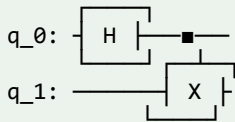
Construct the Bell State circuit

```
# Create a quantum circuit with 2 qubits
qc = QuantumCircuit(2)

# Apply Hadamard gate on qubit 0 to create superposition
qc.h(0)

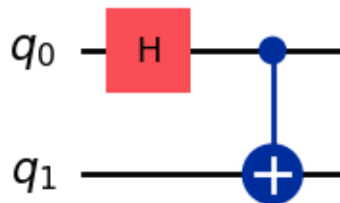
# Apply CNOT gate to entangle qubit 0 (control) and qubit 1 (target)
qc.cx(0, 1)

# Display the circuit diagram
print(qc.draw())
```



Display the circuit diagram graphically

```
qc.draw('mpl')
```



Observe the entangled Bell state

```
state = Statevector.from_instruction(qc)
print("Bell State:", state)
```

```
Bell State: Statevector([0.70710678+0.j, 0.          +0.j, 0.          +0.j,
                        0.70710678+0.j],
                        dims=(2, 2))
```

Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed output verifies the expected quantum behaviour described in the aim of this experiment.

Program 6: Measurement of a Single Qubit

Aim:

To write a Qiskit program to measure a single qubit and display the output.

Code:

Import required libraries

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
```

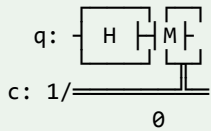
Create a circuit with a qubit and classical bit, then measure

```
# Create a quantum circuit with 1 qubit and 1 classical bit
qc = QuantumCircuit(1, 1)

# Apply Hadamard gate to put the qubit in superposition
qc.h(0)

# Measure qubit 0 into classical bit 0
qc.measure(0, 0)

print(qc.draw())
```



Simulate the circuit using AerSimulator

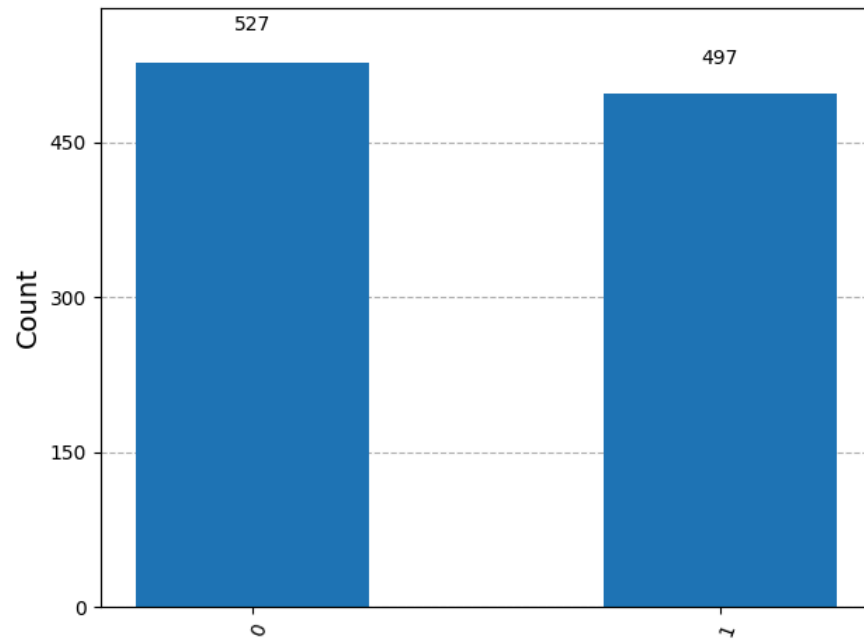
```
simulator = AerSimulator()
job = simulator.run(qc, shots=1024)
result = job.result()
counts = result.get_counts()

print("Measurement counts:", counts)
```

```
Measurement counts: {'0': 527, '1': 497}
```

Display the output as a histogram

```
plot_histogram(counts)
```



Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed output verifies the expected quantum behaviour described in the aim of this experiment.

Program 7: Measurement of a Two-Qubit Circuit

Aim:

To create a quantum circuit with two qubits and perform measurements on both qubits.

Code:

Import required libraries

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
```

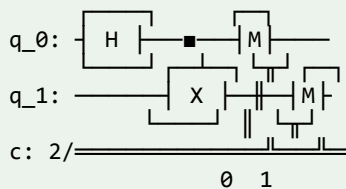
Create a two-qubit circuit and measure both qubits

```
# Create a quantum circuit with 2 qubits and 2 classical bits
qc = QuantumCircuit(2, 2)

# Apply Hadamard on qubit 0 and CNOT (0 -> 1) to entangle both qubits
qc.h(0)
qc.cx(0, 1)

# Measure both qubits
qc.measure([0, 1], [0, 1])

print(qc.draw())
```



Simulate the circuit using AerSimulator

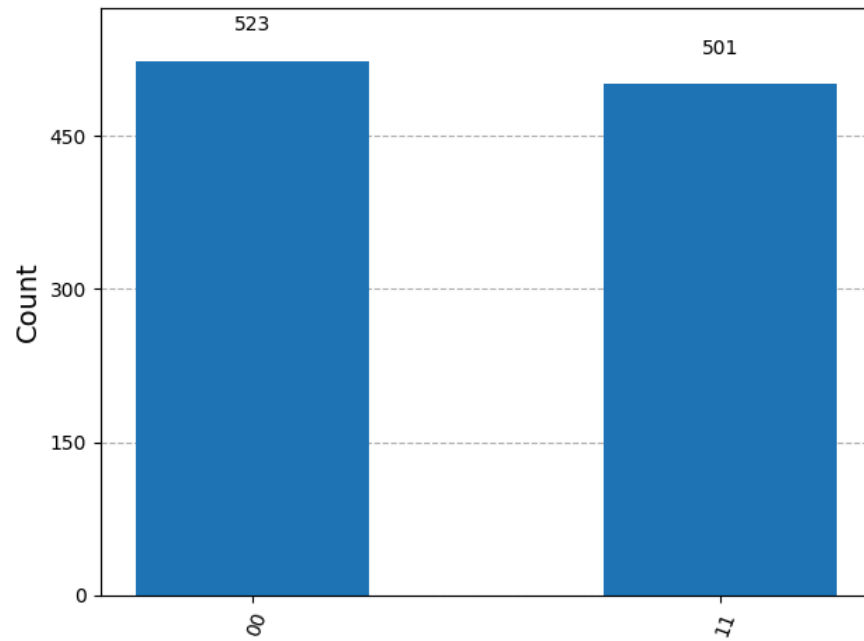
```
simulator = AerSimulator()
job = simulator.run(qc, shots=1024)
result = job.result()
counts = result.get_counts()

print("Measurement counts:", counts)
```

```
Measurement counts: {'00': 523, '11': 501}
```

Display the output as a histogram

```
plot_histogram(counts)
```



Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed output verifies the expected quantum behaviour described in the aim of this experiment.

Program 8: Quantum Circuit with X, H, and CNOT Gates

Aim:

To design and visualize a quantum circuit containing X, H, and CNOT gates.

Code:

Import required libraries

```
from qiskit import QuantumCircuit
```

Design a circuit using X, H, and CNOT gates

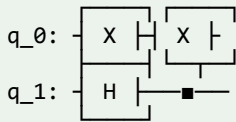
```
# Create a quantum circuit with 2 qubits
qc = QuantumCircuit(2)

# Apply Pauli-X gate on qubit 0
qc.x(0)

# Apply Hadamard gate on qubit 1
qc.h(1)

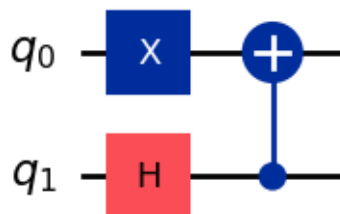
# Apply CNOT gate: qubit 1 (control) -> qubit 0 (target)
qc.cx(1, 0)

# Display the circuit diagram
print(qc.draw())
```



Visualize the circuit graphically

```
qc.draw('mpl')
```



Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed output verifies the expected quantum behaviour described in the aim of this experiment.

Program 9: Simulation using Aer Simulator

Aim:

To simulate a quantum circuit using the Aer Simulator and analyze the measurement results.

Code:

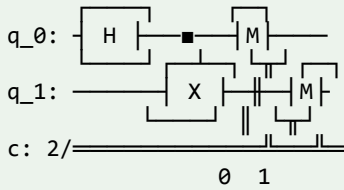
Import required libraries

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
```

Construct a Bell state circuit to simulate

```
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

print(qc.draw())
```



Run the simulation on the Aer Simulator

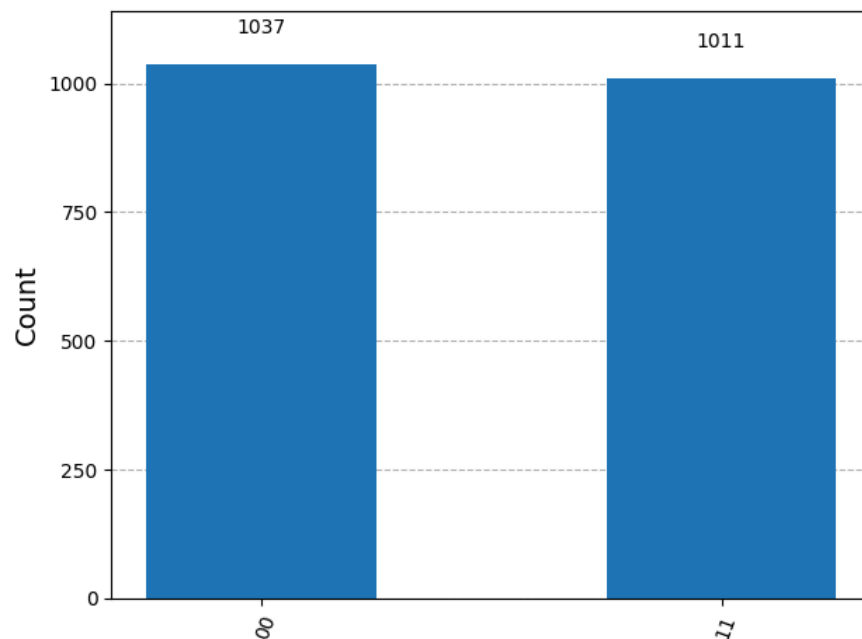
```
simulator = AerSimulator()
job = simulator.run(qc, shots=2048)
result = job.result()
counts = result.get_counts()

print("Measurement counts:", counts)
```

```
Measurement counts: {'11': 1011, '00': 1037}
```

Analyze the results

```
plot_histogram(counts)
```



Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed output verifies the expected quantum behaviour described in the aim of this experiment.

Program 10: Classical Bits vs Quantum Bits

Aim:

To compare the behavior of classical bits and quantum bits using simple Qiskit circuits.

Code:

Import required libraries

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
```

Classical-like behavior: qubit without superposition

```
# A qubit that is never put into superposition behaves like a classical bit.
# It always yields a deterministic (single) outcome upon measurement.
qc_classical = QuantumCircuit(1, 1)
qc_classical.measure(0, 0) # qubit stays in |0>, deterministic result

simulator = AerSimulator()
counts_classical = simulator.run(qc_classical, shots=1024).result().get_counts()
print("Classical-like bit measurement counts:", counts_classical)
```

```
Classical-like bit measurement counts: {'0': 1024}
```

Quantum behavior: qubit in superposition

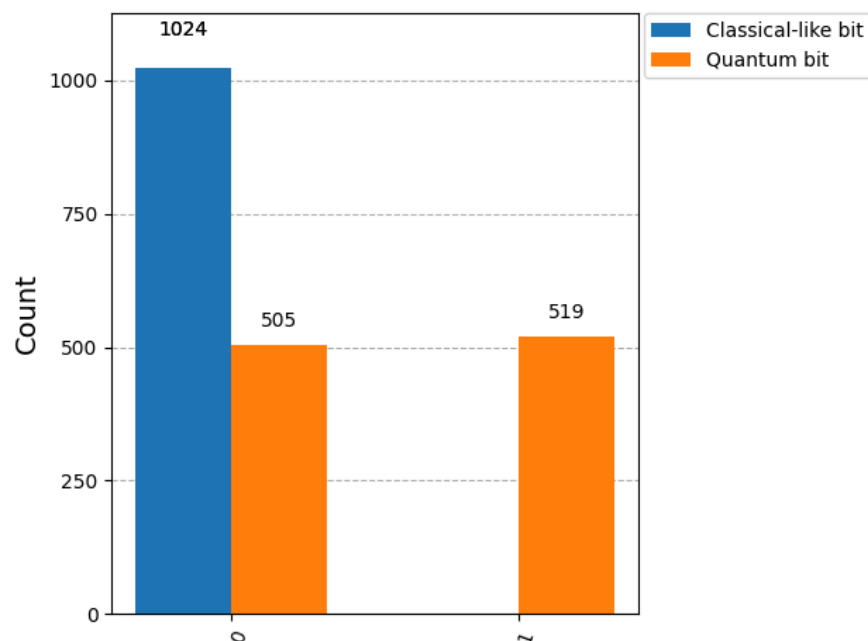
```
# Applying a Hadamard gate creates superposition, giving a probabilistic outcome.
qc_quantum = QuantumCircuit(1, 1)
qc_quantum.h(0)
qc_quantum.measure(0, 0)

counts_quantum = simulator.run(qc_quantum, shots=1024).result().get_counts()
print("Quantum bit (qubit) measurement counts:", counts_quantum)
```

```
Quantum bit (qubit) measurement counts: {'1': 519, '0': 505}
```

Compare the two results side by side

```
plot_histogram([counts_classical, counts_quantum], legend=['Classical-like bit', 'Quantum bit'])
```



Result:

The above quantum circuit was successfully implemented and simulated using Qiskit, and the observed

output verifies the expected quantum behaviour described in the aim of this experiment.

VIVA Q&A:

1. What is Quantum Computing?

Quantum computing is a computing paradigm that uses principles of quantum mechanics to process information using **quantum bits (qubits)**

2. What is the difference between a classical bit and a qubit?

A classical bit can exist only as **0 or 1**, whereas a qubit can exist as $|0\rangle$, $|1\rangle$, or a **superposition of both states**.

3. What is superposition in quantum computing?

Superposition is the property that allows a qubit to exist in a combination of multiple states simultaneously, such as $|0\rangle$ and $|1\rangle$.

4. Write the mathematical representation of a qubit.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where **α and β are complex probability amplitudes**.

5. What is the normalization condition of a qubit?

$$|\alpha|^2 + |\beta|^2 = 1$$

This ensures that the total probability of measuring the qubit is **1**.

6. What is the function of the Pauli-X gate?

The Pauli-X gate acts like a **quantum NOT gate**. It flips the state:

$$|0\rangle \leftrightarrow |1\rangle$$

Your worksheet uses **`qc.x(0)`** to apply this gate

7. What is the purpose of the Hadamard gate?

The Hadamard gate is commonly used to create **superposition** from a computational basis state.

8. What is a Controlled-NOT (CNOT) gate?

CNOT is a two-qubit gate consisting of a **control qubit and a target qubit**. The NOT operation is applied to the target depending on the state of the control qubit.

9. What is quantum entanglement?

Quantum entanglement is a phenomenon in which two or more qubits become correlated, such that the state of one qubit is related to the state of another. The

worksheet demonstrates this using a **Hadamard gate followed by a CNOT gate to create a Bell state**.

10. What is Qiskit and why is it used in quantum computing?

Qiskit is an **open-source quantum computing software development framework developed by IBM**. It is used to design, simulate, visualize, and execute quantum circuits on simulators and real quantum hardware.

RESULT:

Thus, the fundamental concepts of quantum computing were successfully studied and implemented using Qiskit. Quantum circuits involving single and multiple qubits, Pauli-X, Hadamard, and CNOT gates were successfully designed and simulated. Superposition was verified through probabilistic measurement outcomes, and quantum entanglement was demonstrated using the Bell state, where correlated **00** and **11** results were obtained. The behavior of classical bits and quantum bits was also successfully compared using simulation.